

EmprestaAí: Plataforma de empréstimos peer-to-peer

1. Contexto

O mercado de empréstimos peer-to-peer (P2P) tem crescido no Brasil como alternativa ao sistema bancário tradicional, mas ainda carece de soluções que garantam transparência e segurança nas transações entre pessoas. Dentro do Hackathon da QI Tech, surge a oportunidade de desenvolver uma plataforma P2P que aproveite a infraestrutura de Banking as a Service (BaaS) para formalizar acordos de empréstimo, facilitar o acompanhamento de pagamentos e oferecer análise de risco baseada em histórico. Esta solução visa democratizar o acesso ao crédito e promover relações financeiras mais saudáveis e transparentes entre pessoas, eliminando intermediários financeiros e possibilitando condições mais vantajosas para ambas as partes.

2. Objetivo

Desenvolver uma plataforma peer-to-peer de empréstimos que permita aos usuários emprestar e tomar emprestado dinheiro de forma segura e transparente, integrando-se com a infraestrutura da QI Tech para processamento de pagamentos via Pix e gestão de contas digitais. A solução oferecerá funcionalidades de formalização de contratos, análise de crédito baseada em histórico de transações e sistema de score de confiabilidade, proporcionando uma experiência completa que promove relacionamentos financeiros saudáveis entre pessoas e democratiza o acesso ao crédito fora do sistema bancário tradicional.

3. Estrutura de Banco de Dados

Para suportar a plataforma peer-to-peer de empréstimos, propomos uma estrutura de banco de dados relacional robusta, otimizada para gestão de contratos financeiros, histórico de transações e análise de risco. As principais entidades e seus relacionamentos são detalhados abaixo, visando garantir a integridade dos dados, rastreabilidade completa das operações, segurança das informações financeiras e escalabilidade para suportar o crescimento da base de usuários. A arquitetura contempla mecanismos de auditoria, versionamento de contratos e armazenamento de evidências digitais para assegurar a conformidade legal e a confiança entre as partes envolvidas nas transações de empréstimo.

Diagrama de Entidade-Relacionamento (DER)

erDiagram

```
-- =====
-- Tipos enumerados
-- =====
CREATE TYPE person_type AS ENUM ('individual','company');
CREATE TYPE application_status AS ENUM (
    'draft','submitted','kyc_failed','rejected','under_review',
    'approved','offer_sent','accepted','withdrawn'
);
CREATE TYPE kyc_status AS ENUM ('pending','passed','failed');
CREATE TYPE underwriting_result AS ENUM ('approved','rejected','manual_review');
CREATE TYPE loan_status AS ENUM ('active','closed','defaulted','prepaid','cancelled');
CREATE TYPE payment_status AS ENUM ('scheduled','paid','late','partial','failed');
CREATE TYPE collateral_type AS ENUM ('vehicle','real_estate','guarantee','other');

-- =====
```

```

-- Customers / Applicants
-- =====
CREATE TABLE customer (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  person_type person_type NOT NULL,
  legal_name TEXT NOT NULL,
  display_name TEXT,
  cpf_cnpj VARCHAR(20) UNIQUE,
  dob DATE,
  email TEXT,
  phone VARCHAR(30),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  deleted_at TIMESTAMP WITH TIME ZONE NULL
);

-- =====
-- Addresses
-- =====
CREATE TABLE address (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID REFERENCES customer(id) ON DELETE CASCADE,
  type VARCHAR(20), -- ex: 'residential', 'billing'
  street TEXT,
  city TEXT,
  state VARCHAR(50),
  postal_code VARCHAR(20),
  country VARCHAR(50) DEFAULT 'BR',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Income sources / comprovação de renda
-- =====
CREATE TABLE income_source (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID REFERENCES customer(id) ON DELETE CASCADE,
  type VARCHAR(50), -- 'salary', 'self_employed', 'pension', etc
  employer TEXT,
  monthly_amount NUMERIC(15,2) NOT NULL,
  proof_documents JSONB, -- paths/metadata
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====

```

```

-- Products (tipos de empréstimo)
-- =====
CREATE TABLE product (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code VARCHAR(50) UNIQUE NOT NULL,
  name TEXT NOT NULL,
  max_term_months INT,
  max_amount NUMERIC(18,2),
  base_interest_rate NUMERIC(6,4),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Application (solicitação)
-- =====
CREATE TABLE application (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID REFERENCES customer(id) ON DELETE RESTRICT,
  product_id UUID REFERENCES product(id),
  amount_requested NUMERIC(18,2) NOT NULL,
  term_months INT,
  purpose TEXT,
  status application_status DEFAULT 'draft',
  dti_estimated NUMERIC(5,2), -- porcentagem estimada
  bureau_score INT,
  offer JSONB, -- oferta calculada (parcelas, juros, CET)
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Documents uploaded
-- =====
CREATE TABLE document (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES application(id) ON DELETE CASCADE,
  customer_id UUID REFERENCES customer(id) ON DELETE CASCADE,
  doc_type VARCHAR(50), -- 'id', 'proof_of_income', etc
  storage_path TEXT,
  metadata JSONB,
  verified BOOLEAN DEFAULT FALSE,
  uploaded_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====

```

```

-- KYC / AML checks (histórico)
-- =====
CREATE TABLE kyc_check (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES application(id) ON DELETE CASCADE,
  customer_id UUID REFERENCES customer(id),
  status kyc_status DEFAULT 'pending',
  provider VARCHAR(100), -- ex: 'Serasa', 'KYCProviderX'
  result JSONB,
  checked_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Credit bureau checks
-- =====
CREATE TABLE credit_check (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES application(id) ON DELETE CASCADE,
  provider VARCHAR(100),
  score INT,
  report JSONB,
  fetched_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Staff / Analistas
-- =====
CREATE TABLE staff (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(100) UNIQUE,
  full_name TEXT,
  role VARCHAR(50),
  email TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Underwriting / decisões (automáticas ou manuais)
-- =====
CREATE TABLE underwriting (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES application(id) ON DELETE CASCADE,
  staff_id UUID REFERENCES staff(id) NULL,
  result underwriting_result NOT NULL,
  reason TEXT,
  rules_triggered JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====

```

```

-- Loans (contratos efetivados)
-- =====
CREATE TABLE loan (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES application(id) UNIQUE, -- 1:1 application->loan
  customer_id UUID REFERENCES customer(id),
  product_id UUID REFERENCES product(id),
  principal NUMERIC(18,2) NOT NULL,
  outstanding_balance NUMERIC(18,2) NOT NULL,
  interest_rate NUMERIC(7,4),
  term_months INT,
  start_date DATE,
  maturity_date DATE,
  status loan_status DEFAULT 'active',
  amortization_method VARCHAR(50) DEFAULT 'price', -- price, sac, etc
  repayments JSONB, -- snapshot or extra rules
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Loan schedule (parcelas previstas)
-- =====
CREATE TABLE loan_schedule (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id UUID REFERENCES loan(id) ON DELETE CASCADE,
  installment_no INT NOT NULL,
  due_date DATE NOT NULL,
  principal_amount NUMERIC(18,2) NOT NULL,
  interest_amount NUMERIC(18,2) NOT NULL,
  fees_amount NUMERIC(18,2) DEFAULT 0,
  total_amount NUMERIC(18,2) NOT NULL,
  status payment_status DEFAULT 'scheduled',
  paid_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  UNIQUE (loan_id, installment_no)
);

-- =====
-- Payments (recebidos)
-- =====
CREATE TABLE payment (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id UUID REFERENCES loan(id) ON DELETE CASCADE,
  schedule_id UUID REFERENCES loan_schedule(id) NULL,
  amount NUMERIC(18,2) NOT NULL,
  provider_transaction_id TEXT,
  status payment_status DEFAULT 'paid',
  posted_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  metadata JSONB
);

-- =====

```

```

-- Collaterals / garantias
-- =====
CREATE TABLE collateral (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id UUID REFERENCES loan(id) ON DELETE CASCADE,
  type collateral_type,
  description TEXT,
  value_estimate NUMERIC(18,2),
  registration JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Guarantors (avalistas)
-- =====
CREATE TABLE guarantor (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id UUID REFERENCES loan(id) ON DELETE CASCADE,
  customer_id UUID REFERENCES customer(id) NULL,
  name TEXT,
  cpf_cnpj VARCHAR(20),
  contact JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Collection actions / histórico de cobrança
-- =====
CREATE TABLE collection_action (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id UUID REFERENCES loan(id) ON DELETE CASCADE,
  action_type VARCHAR(50), -- 'reminder','call','legal','restructure'
  notes TEXT,
  performed_by UUID REFERENCES staff(id) NULL,
  performed_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====
-- Audit log (escrever via triggers / app)
-- =====
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  entity_type TEXT NOT NULL,
  entity_id UUID,
  action VARCHAR(20) NOT NULL, -- 'insert','update','delete'
  actor TEXT,
  changes JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- =====

```

```

-- Payment methods, transactions e merchants (conforme imagem)
-- =====
CREATE TABLE payment_methods (
  payment_method_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES customer(id) ON DELETE CASCADE,
  details VARCHAR(255),
  is_default BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

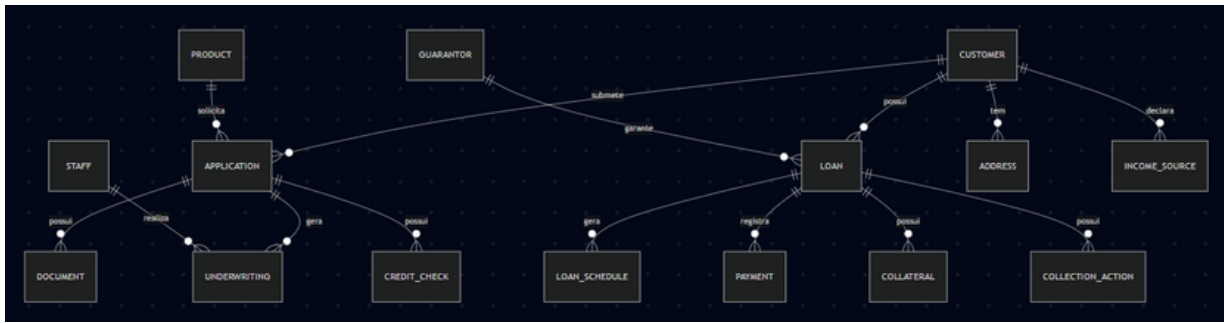
CREATE TABLE merchants (
  merchant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  cnpj VARCHAR(20),
  email VARCHAR(255),
  phone_number VARCHAR(50),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE TABLE merchant_accounts (
  merchant_account_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  merchant_id UUID REFERENCES merchants(merchant_id) ON DELETE CASCADE,
  account_identifier VARCHAR(255), -- ex: chave PIX, conta bancária
  balance DECIMAL(18,2) DEFAULT 0,
  currency VARCHAR(10) DEFAULT 'BRL',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE TABLE transactions (
  transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_account_id UUID REFERENCES merchant_accounts(merchant_account_id) NULL,
  receiver_account_id UUID REFERENCES merchant_accounts(merchant_account_id) NULL,
  amount DECIMAL(18,2) NOT NULL,
  currency VARCHAR(10) DEFAULT 'BRL',
  status VARCHAR(50),
  transaction_type VARCHAR(100), -- 'payment','refund','transfer'
  description VARCHAR(500),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

-- Índices úteis
-- =====
CREATE INDEX idx_application_customer_status ON application(customer_id, status);
CREATE INDEX idx_loan_status_balance ON loan(status, outstanding_balance);
CREATE INDEX idx_schedule_due_date ON loan_schedule(due_date);
CREATE INDEX idx_customer_cpf ON customer(cpf_cnpj);
CREATE INDEX idx_transactions_sender_receiver ON transactions(sender_account_id, receiver_account_id);
CREATE INDEX idx_merchant_account_merchant ON merchant_accounts(merchant_id);

```



Descrição das Entidades

USERS

Guarda os dados do usuário da plataforma.

- `user_id`: identificador do usuário (usado internamente).
- `whatsapp_id`: ID do WhatsApp (para integração e identificação).
- `name`: nome do usuário.
- `email`: e-mail de contato.
- `phone_number`: telefone.
- `created_at` / `updated_at`: quando o cadastro foi criado/atualizado.

ACCOUNTS

Representa a “carteira” financeira de cada usuário — onde fica o saldo disponível.

- `account_id`: identificador da conta.
- `user_id`: liga a conta ao usuário dono.
- `balance`: saldo atual da conta.
- `currency`: moeda usada (ex.: BRL).
- `created_at` / `updated_at`: controle de criação/alteração.

PAYMENT_METHODS

Métodos de pagamento que o usuário adiciona (Pix, cartão etc.). Serve para pagar/receber.

- `payment_method_id`: identificador do método.
- `user_id`: proprietário do método.
- `type`: tipo (ex.: PIX, CREDIT_CARD).
- `details`: informações úteis (ex.: última4 do cartão ou chave Pix).
- `is_default`: indica se é o método padrão do usuário.
- `created_at` / `updated_at`: histórico.

TRANSACTIONS

Registra todas as movimentações (envio, recebimento, depósito, saque). É o histórico financeiro.

- `transaction_id`: identificador da transação.
- `sender_account_id` / `receiver_account_id`: de quem saiu e para quem entrou.
- `amount`: valor movimentado.
- `currency`: moeda da transação.
- `status`: situação atual (ex.: pendente, concluída, falhada).
- `transaction_type`: motivo/tipo (envio, recebimento, depósito, saque).
- `description`: anotação sobre a transação.
- `created_at` / `updated_at`: quando ocorreu/foi atualizado.

MERCHANTS

Dados de comerciantes/parceiros que recebem pagamentos pelo WhatsApp.

- merchant_id: identificador do comerciante.
- name: nome do estabelecimento.
- cnpj: cadastro da empresa (quando aplicável).
- email / phone_number: contatos.
- created_at / updated_at: histórico do cadastro.

Considerações de Arquitetura

- Segurança: Implementação de criptografia de ponta a ponta para dados sensíveis, autenticação multifator e conformidade com regulamentações financeiras (LGPD, PCI DSS).
- Escalabilidade: Utilização de bancos de dados distribuídos ou soluções de nuvem que permitam o crescimento horizontal.
- Consistência: Garantia de consistência transacional (ACID) para todas as operações financeiras.
- Privacidade: Mínima exposição de dados pessoais e controle de permissões para acesso restrito a informações sensíveis.

4. Estrutura de Front-end

A plataforma peer-to-peer de empréstimos requer uma interface web moderna e intuitiva para administração, gestão de contratos e acompanhamento de transações, além de uma integração futura com aplicativos móveis. A estrutura de front-end proposta utiliza tecnologias modernas para garantir uma experiência fluida, segura e responsiva.

Arquitetura do Front-end

A aplicação front-end foi desenvolvida utilizando React.js com TypeScript, proporcionando uma base sólida para desenvolvimento escalável e manutenível. A arquitetura segue o padrão de componentes reutilizáveis e gerenciamento de estado centralizado para garantir consistência e performance.

Tecnologias Principais

- React.js 18+: Framework principal para construção da interface de usuário
- TypeScript: Para tipagem estática e melhor experiência de desenvolvimento
- Tailwind CSS: Framework CSS utilitário para estilização rápida e consistente
- shadcn/ui: Biblioteca de componentes acessíveis e customizáveis
- React Router: Para navegação client-side e gerenciamento de rotas
- React Hook Form: Para gerenciamento eficiente de formulários
- Zod: Para validação de dados e schemas
- TanStack Query: Para gerenciamento de estado assíncrono e cache de dados da API
- Recharts: Para visualização de dados e gráficos financeiros
- Lucide React: Para ícones consistentes e modernos
- date-fns: Para manipulação e formatação de datas

```
emprestaai-connect/src/
├── assets
│   ├── icon.svg
│   ├── logo.png
│   └── logo.svg
├── components
│   ├── ui
│   │   ├── accordion.tsx
│   │   ├── alert-dialog.tsx
│   │   ├── alert.tsx
│   │   ├── aspect-ratio.tsx
│   │   ├── avatar.tsx
│   │   ├── badge.tsx
│   │   ├── breadcrumb.tsx
│   │   ├── button.tsx
│   │   └── calendar.tsx
│   ...
├── pages
│   ├── Cadastro.tsx
│   ├── Home.tsx
│   ├── Login.tsx
│   ├── NotFound.tsx
│   ├── QueroEmprestar.tsx
│   └── QueroReceber.tsx
├── App.css
├── App.tsx
├── index.css
├── main.tsx
└── vite-env.d.ts
```

Principais Funcionalidades da Interface

Dashboard do Usuário

O dashboard principal oferece uma visão geral completa das atividades financeiras do usuário, incluindo empréstimos ativos (como credor e devedor), histórico de transações, score de confiabilidade e métricas de performance. Inclui gráficos interativos mostrando volume de empréstimos, taxas de inadimplência, rendimentos obtidos e tendências temporais.

Gerenciamento de Empréstimos

Interface para criar, visualizar e gerenciar empréstimos na plataforma. Permite ao usuário solicitar empréstimos definindo valor, prazo e condições, ou ofertar crédito para outros usuários. Inclui visualização detalhada de contratos, cronograma de parcelas, status de pagamentos e histórico completo de cada transação.

Análise de Crédito e Score

Score de confiabilidade pessoal baseado no histórico de transações, cálculo de juros com base no score através de um modelo de ML. Interface para consultar o score de potenciais tomadores antes de conceder empréstimos, com detalhamento dos fatores que compõem a pontuação e recomendações para melhoria.

Painel de transações

Visualização detalhada de todas as transações realizadas na plataforma, com filtros por data, status, valor e tipo de operação. Inclui funcionalidades de busca avançada, exportação de dados e geração de comprovantes para fins de controle pessoal.

Configurações e Perfil

Interface para gerenciar dados pessoais, informações bancárias, preferências de notificação e configurações de segurança. Inclui funcionalidades de verificação de identidade, vinculação de contas bancárias via QI Tech e gestão de métodos de pagamento (Pix).

Relatórios e Análises

Ferramentas de geração de relatórios financeiros personalizados, incluindo demonstrativos de rendimentos, histórico de inadimplência, projeções de ganhos e análises comparativas de performance ao longo do tempo.

Sistema de Notificações

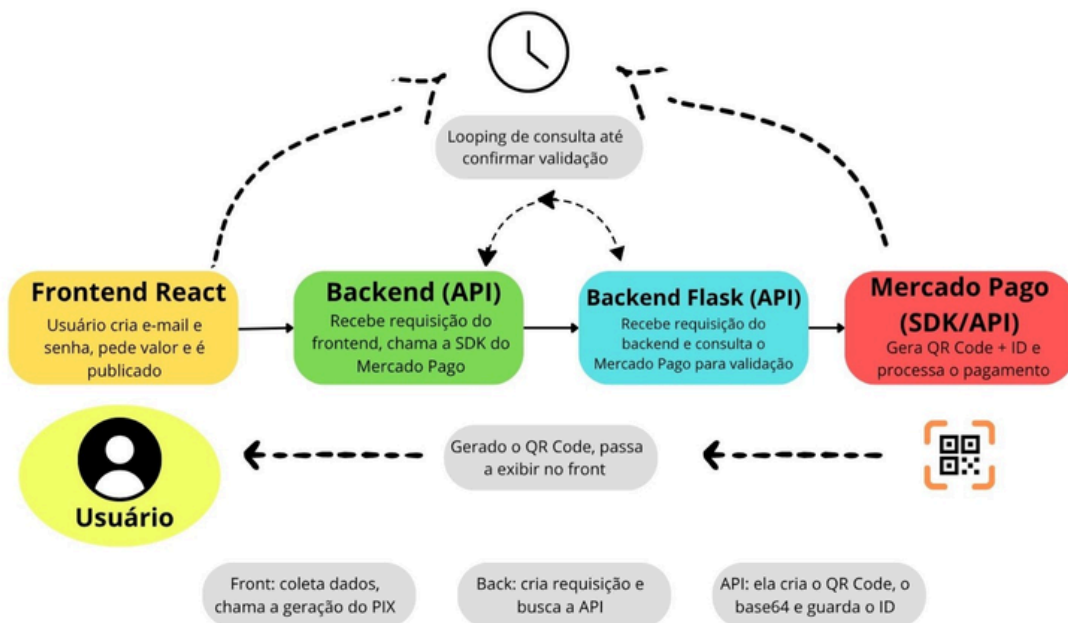
Central de notificações em tempo real alertando sobre vencimentos de parcelas, solicitações de empréstimo, confirmações de pagamento e atualizações de score. Permite configuração personalizada de alertas via email.

Segurança no Front-end

- Autenticação JWT: Tokens seguros para autenticação de usuários
- Validação de entrada: Validação rigorosa de todos os dados inseridos
- Autenticação 2FA: Camada adicional de segurança via aplicativo autenticador (TOTP)
- HTTPS obrigatório: Todas as comunicações criptografadas

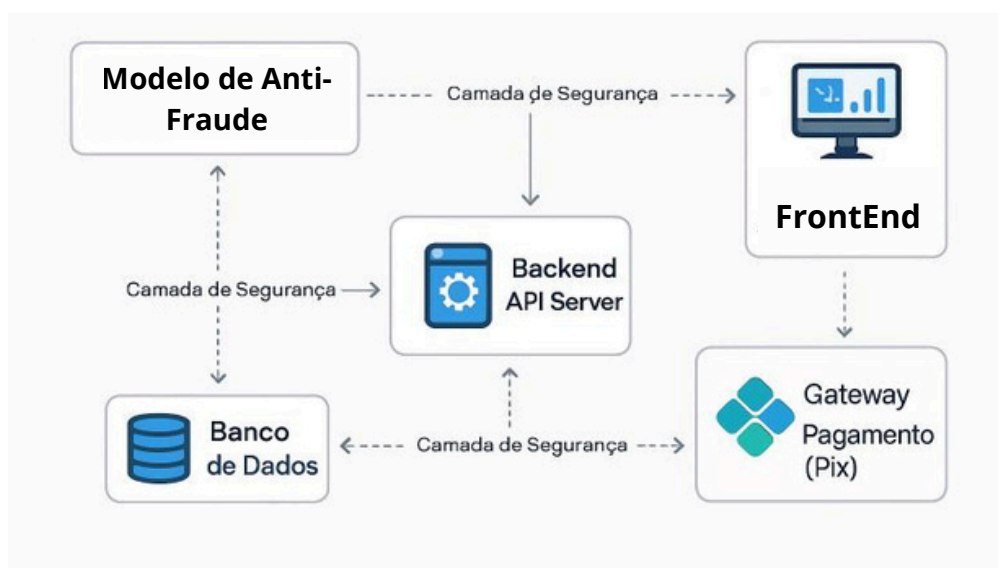
5. Imagens Ilustrativas para Fluxos e Integrações

Fluxo de funcionamento da plataforma



O fluxo segue uma sequência onde o usuário ao se cadastrar na plataforma, passa por uma verificação, após esse procedimento, ele deve escolher se quer emprestar ou pedir emprestado, caso peça emprestado, deve aguardar alguém aceitar lhe emprestar, já caso queira emprestar, deve escolher uma oferta e aceitar, gerando o qr code pix para pagamento e finalização da transação.

Arquitetura do Sistema



Conectando-se ao Backend por meio de camadas de autenticação e criptografia. O servidor backend gerencia toda a lógica de negócio, incluindo cadastro, validação de identidade e análise de risco, integrando-se ao banco de dados para persistência das informações. Para o processamento das transações financeiras, a aplicação se conecta ao Gateway de Pagamento (Pix). Além disso, os módulos de antifraude garantem a rastreabilidade e a conformidade regulatória da plataforma.

Interface da Plataforma



O dashboard possui uma interface intuitiva, permitindo ao usuário visualizar métricas como o número de solicitações ativas, o score médio de seus devedores, o valor total que ele já emprestou, dentre outras.

Fluxos de Integração

Integração com API de score de crédito

A integração com uma API de score de crédito permite que o sistema busque pela confiabilidade do usuário, para saber se ele paga suas dívidas em dia, através disso, permitindo o cálculo do juros do seu empréstimo com base nesses índices

Integração com API de Pagamento Pix

O sistema integra-se com provedores de pagamento que suportam Pix, permitindo a geração de QR Codes dinâmicos, processamento de pagamentos instantâneos e recebimento de webhooks de confirmação. Todas as transações são processadas de acordo com as regulamentações do Banco Central do Brasil.

Segurança dos Dados

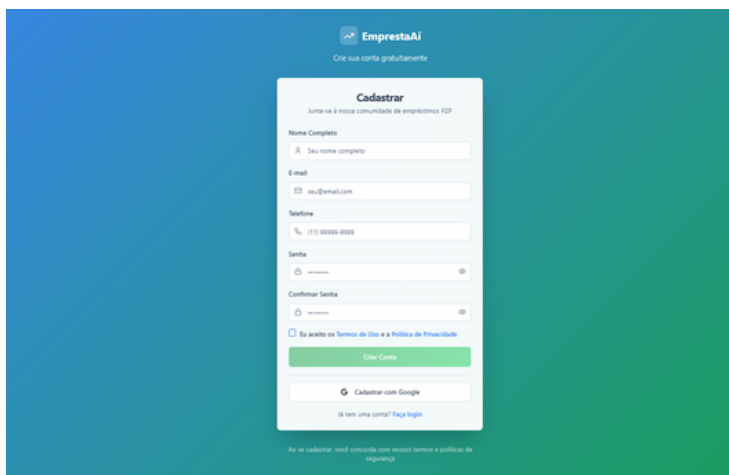
Nossa plataforma possui vários recursos de segurança como autenticação de 2 fatores (2FA), checagem de documentos e veracidade de informações no momento do cadastro, todos dados trafegam com criptografia, dentre outros recursos

Principais Funcionalidades do Backend:

- **API Pix:** Gera o qrcode de pagamento para o empréstimo
- **Modelo de ML:** Calcula o juros com base no score de crédito
- **Anti-fraude:** Realiza a checagem do usuário no momento do cadastro
- **Conexão com o banco de dados**
- **Segurança:** Validação de tokens, criptografia de dados sensíveis, 2FA e hash

Exemplo de Fluxo Completo

1. Usuário se Cadastra na Plataforma



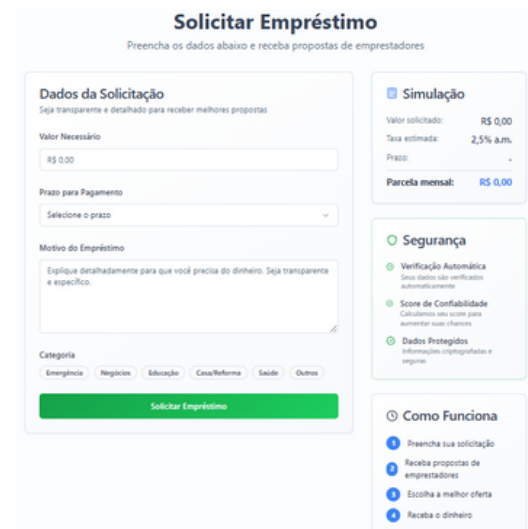
The screenshot shows the 'Cadastrar' (Register) page of the EmprestaAI platform. The page has a teal background. At the top, it says 'EmprestaAI' and 'Crie sua conta gratuitamente'. The registration form is a white card with the following fields: 'Nome Completo' (Full Name), 'E-mail', 'Telefone' (Phone), 'Senha' (Password), and 'Confirmar Senha' (Confirm Password). There is a checkbox for 'Eu aceito os Termos de Uso e a Política de Privacidade'. Below the form is a green 'Criar Conta' button. At the bottom of the card, there are links for 'Cadastrar com Google' and 'Já tem uma conta? Faça login'. At the very bottom of the page, there is a small note: 'Ao se cadastrar, você concorda com nossos termos e política de privacidade'.

2. Seu cadastro passa por um processo de aprovação

3. Caso aprovado, agora ele emprestar ou pedir emprestado



The screenshot shows the 'Oportunidades de Empréstimo' (Loan Opportunities) page. At the top, it says 'Oportunidades de Empréstimo' and 'Encontre pessoas que precisam de ajuda financeira e faça a diferença'. Below this, there are three summary cards: 'Total Solicitado' (Total Requested) for R\$ 26.500,00, 'Solicitações Ativas' (Active Requests) for 6, and 'Score Médio' (Average Score) for 735. Below these is a search bar with the text 'Buscar por nome, motivo ou valor...' and buttons for 'Filtros' and 'Buscar'. Below the search bar are several tabs: 'Todos', 'Score Alto (800+)', 'Até R\$ 1.000', 'Emergência', and 'Negócios'. Below the tabs is a grid of loan opportunities. Each opportunity card shows the name of the person, their profile picture, their score, the amount they need, the interest rate, the term, and a brief description of their reason for borrowing. For example, Maria Silva needs R\$ 5.000,00 at 2.5% a.m. for 12 months to improve her family's living conditions. Below each card are buttons for 'Ver Detalhes' and 'Ver Perfil'.



The screenshot shows the 'Solicitar Empréstimo' (Request Loan) page. At the top, it says 'Solicitar Empréstimo' and 'Preencha os dados abaixo e receba propostas de emprestadores'. The page is divided into three main sections. The left section is 'Dados da Solicitação' (Loan Request Data), which includes fields for 'Valor Necessário' (Amount Needed), 'Prazo para Pagamento' (Payment Term), and 'Motivo do Empréstimo' (Reason for Loan). The middle section is 'Simulação' (Simulation), which shows the 'Valor solicitado' (Amount requested) as R\$ 0,00, the 'Taxa estimada' (Estimated rate) as 2,5% a.m., and the 'Parcela mensal' (Monthly installment) as R\$ 0,00. The right section is 'Segurança' (Security), which includes 'Verificação Automática' (Automatic verification), 'Score de Confiabilidade' (Reliability score), and 'Dados Protegidos' (Protected data). At the bottom of the page is a green 'Solicitar Empréstimo' button. Below the button is a section titled 'Como Funciona' (How it works) with four steps: 1. Preencha sua solicitação (Fill out your request), 2. Receba propostas de emprestadores (Receive proposals from lenders), 3. Escolha a melhor oferta (Choose the best offer), and 4. Receba o dinheiro (Receive the money).

4. Agora basta esperar alguém aceitar sua oferta, receber o dinheiro e depois pagar de volta

Segurança e Conformidade

Nosso sistema possui diversas camadas de segurança, incluindo criptografia de ponta a ponta e validação de tokens. Atende às regulamentações e está em conformidade com a LGPD para proteção de dados pessoais.

Escalabilidade e Performance

A arquitetura da plataforma EmprestaAí foi projetada para suportar crescimento contínuo, garantindo alta performance e disponibilidade mesmo com milhares de operações financeiras simultâneas.

Próximos passos

Integração entre FrontEnd e BackEnd

Implementação do mecanismo de Anti-Fraude

Implementação de ML para cálculo dos juros

Revisar toda a aplicação, visando e garantindo maior segurança